



Vision-Based Robotic Pick-and-Place Using YOLO and RGB-D Sensing

1. System Overview

This document explains how to use and understand a vision-based robotic pick-and-place system developed in the ColorsLab environment. The system allows a robotic arm to autonomously detect an object placed on a table, estimate its position using camera data, and perform a pick-and-place operation.

The system integrates computer vision, geometric localization, and robot motion control into a single pipeline. An RGB camera observes the workspace, a deep learning–based detector identifies the target object, and geometric reasoning is used to estimate the object’s 3D position. The estimated position is then sent to the robot controller to execute the manipulation task.

The goal of this document is to provide a practical guide that enables users to understand, operate, and debug the system in a laboratory setting.

2. Hardware Setup

The system consists of three main hardware components arranged within a shared workspace.

Robotic Manipulator

A 6-DOF xArm robotic manipulator is used to perform pick-and-place operations. The robot is equipped with a parallel gripper and supports Cartesian position control. Communication with the robot is established via Ethernet, allowing direct position commands with predefined speed and acceleration limits.



Vision Sensor

An Intel RealSense RGB-D camera is mounted above the workspace in a fixed top-down configuration. Although the camera provides both RGB and depth streams, the system primarily relies on the RGB stream combined with geometric assumptions about the workspace. Images are captured at a resolution of 640×480 pixels.



Computing Unit

A central workstation runs all vision processing and robot control software. The system is implemented in Python and communicates with the robot using the xArm API.

3. Software Pipeline Overview

The system operates continuously in a closed-loop perception-to-action pipeline. The main stages of the pipeline are:

1. Acquisition of RGB images from the camera
2. Object detection using a deep learning-based model
3. Selection of a representative image point for localization
4. Geometric 2D-to-3D projection using workspace constraints
5. Camera-to-robot coordinate transformation
6. Empirical error correction
7. Safe robot motion execution

Each stage produces an intermediate output that is passed to the next stage. This modular structure allows individual components to be tested and adjusted independently.

4. Object Detection

The object detection model was trained using a custom dataset obtained through the Roboflow platform. The dataset consists of RGB images of lemons captured under laboratory conditions, covering variations in object position, orientation, and lighting.

Object detection is performed using YOLOv4, which provides a good balance between detection accuracy and real-time performance. The detector processes incoming RGB images and outputs two-dimensional bounding boxes around detected objects.

Rather than using the geometric center of the bounding box, the system selects a reference point near the bottom center of the detected object. This point better corresponds to the

physical contact location between the object and the table surface and helps reduce errors caused by perspective distortion.

The selected pixel coordinates serve as the input for geometric localization.

5. 2D-to-3D Localization and Calibration

2D-to-3D Projection

Since the object detector outputs only 2D image coordinates, depth information is inferred geometrically. A viewing ray is constructed from the selected pixel using the camera intrinsic parameters based on the pinhole camera model. This ray is intersected with a predefined planar model representing the table surface.

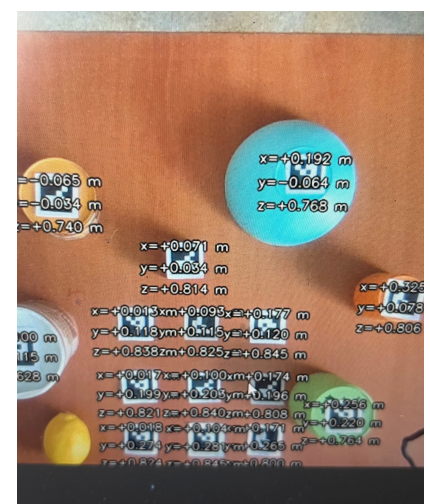
By assuming that the object lies on the table plane, a consistent 3D position estimate is obtained in the camera coordinate frame.

Camera-to-Robot Calibration

The estimated 3D position is transformed into the robot base frame using a rigid body transformation defined by a rotation matrix R and a translation vector t :

$$P(\text{robot}) = R \cdot P(\text{cam}) + t$$

The calibration parameters were obtained offline using multiple points with different heights and positions corresponding to those collected across the workspace. The points were distributed in a grid-like pattern to capture spatial variations. Although rigid point-set alignment methods such as the Kabsch or Umeyama algorithm can be used to compute this transformation, the resulting matrix is treated as fixed during runtime.



6. Empirical Error Compensation

In practice, small systematic localization errors remain even after rigid calibration. These errors are mainly caused by lens distortion, mechanical tolerances, and minor variations in camera placement.

To compensate for these effects, an empirical correction layer is applied to the estimated planar coordinates. A linear regression-based model refines the (X, Y) coordinates to improve accuracy across the workspace. Additionally, a configurable offset term allows quick adjustments to daily alignment changes without repeating the full calibration procedure.

7. Robot Control and Motion Execution

Robot motion is executed using Cartesian position control to ensure smooth, predictable, and safe operation. Once the corrected target position is obtained, the robot follows a predefined pick-and-place sequence.

Each manipulation cycle includes:

- Moving to a pre-grasp hover position above the object
- Vertical descent to the grasp position
- Gripper closure
- Vertical lift to a safe clearance height
- Transport to the drop location
- Controlled release of the object

All motions are executed with predefined speed and acceleration limits. The gripper is controlled explicitly through open and close commands synchronized with the motion

sequence. This strategy ensures collision-free operation even in the presence of small localization errors.

8. Running the System

To operate the system:

1. Ensure that the robot, camera, and workstation are powered on and connected.
2. Launch the Python control script.
3. Verify that the camera feed and detection output are visible.
4. Place the target object on the table surface within the workspace.
5. Once the object is detected and localized, trigger the pick-and-place action (e.g., via keyboard input).

The system continuously updates detections and localization results until the operation is completed or manually stopped.

9. Known Issues and Practical Notes

- The camera mount is not fully rigid, and small mechanical shifts may occur over time.
- As a result, camera-to-robot calibration accuracy can degrade gradually.
- Periodic recalibration of the transformation matrix may be required to maintain reliable performance.
- Empirical correction parameters may need minor adjustment depending on daily setup variations.

These limitations are typical in laboratory environments and should be considered during extended operation.

10. Summary

This document presented a practical guide for operating a vision-based robotic pick-and-place system. By combining deep learning-based object detection, geometric localization, offline calibration, and safety-oriented robot control, the system enables reliable manipulation in a laboratory setting. The modular design allows future extensions such as multi-object handling, automated recalibration, or closed-loop visual feedback.